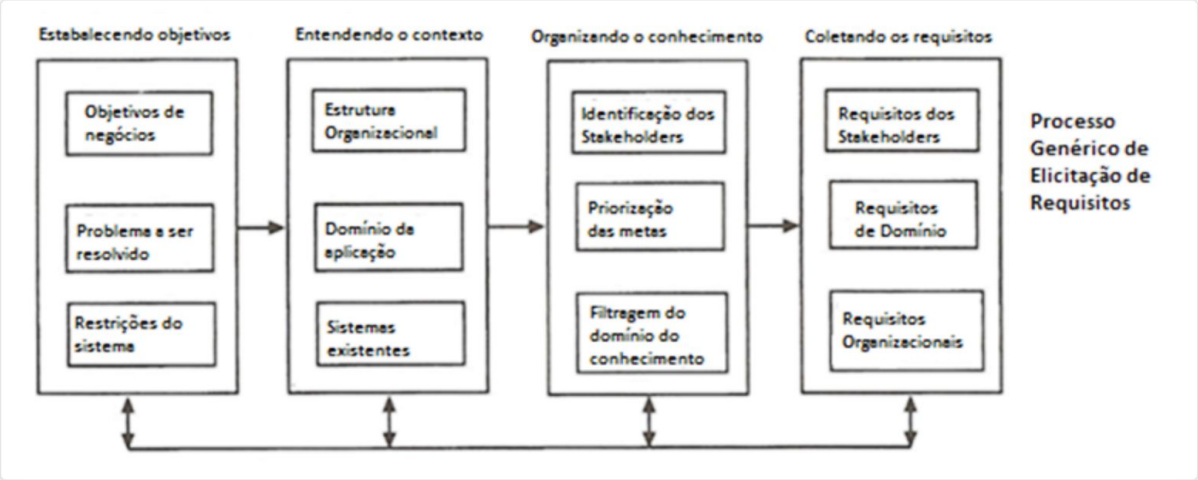
Engenharia e Processos de Software

Prof. Manoel Macedo

Um pouco mais sobre Requisitos!

A **engenharia de requisitos** é uma das etapas mais importantes no desenvolvimento de software. É nela que se identifica o que o sistema deve fazer, quais restrições precisa respeitar e quais necessidades dos usuários devem ser atendidas. Sem essa etapa, há um grande **risco** de construir um produto que não resolve o problema real, gerando desperdício de tempo, dinheiro e esforço.

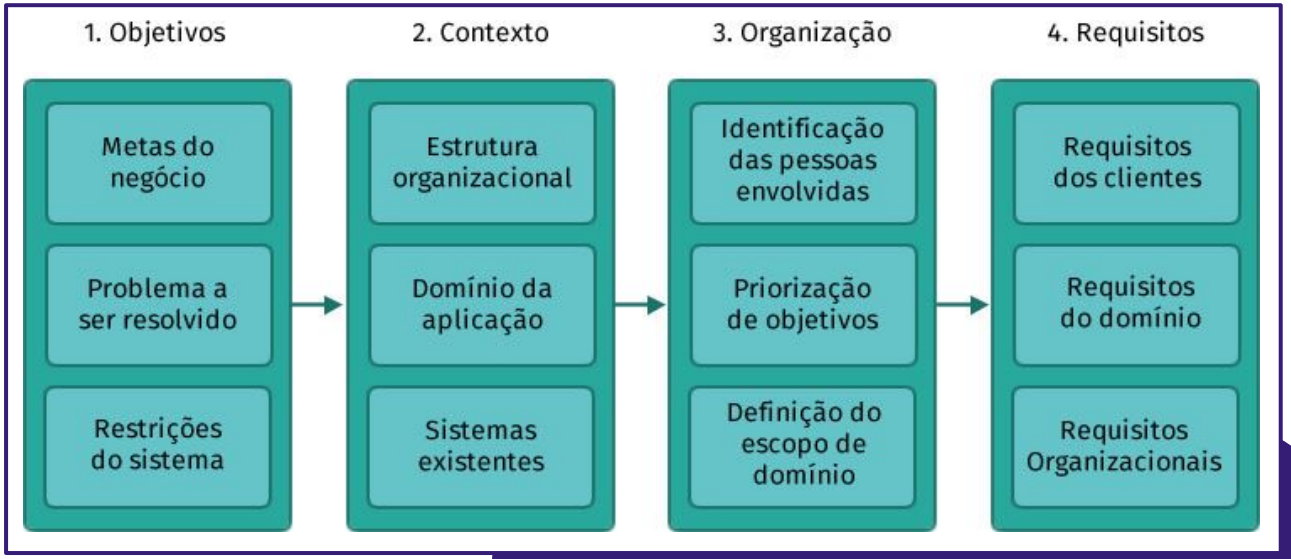
Processo Genérico de Levantamento de Requisitos.



Fonte: Sommerville e Kotonya (1998, p.59)

Um dos primeiros passos é a **elicitação de requisitos**, ou seja, descobrir, levantar e entender quais funcionalidades o sistema deve possuir. Esse processo envolve identificar fontes de informação, como usuários, leis, regulamentos, investidores ou até sistemas anteriores. O objetivo é compreender bem o contexto para depois definir os requisitos de forma clara e organizada.

Um outro exemplo de processo de elicitação de requisitos



Existem diversas **técnicas de elicitação**, cada uma adequada a situações específicas. Entre as mais comuns estão os questionários, as entrevistas e a observação. Questionários são úteis quando há muitos usuários ou quando estão dispersos geograficamente. Entrevistas permitem um diálogo mais profundo com stakeholders, captando detalhes e até informações não verbais. Já a observação é eficaz quando se quer entender como uma tarefa realmente acontece no dia a dia.

Depois de coletar os requisitos, é necessário organizá-los.
Em geral, eles podem ser classificados em três tipos:

- **Requisitos de processo**, que dizem respeito a como o software será desenvolvido;
- **Requisitos de projeto**, ligados a prazos, custos e restrições; e
- **Requisitos de produto**, que correspondem diretamente às funcionalidades que o sistema terá. Essa separação ajuda a equipe a lidar com cada aspecto de forma mais eficiente.

Além de elencar requisitos, também é preciso atribuir **atributos** a eles, como prioridade, risco, dificuldade de implementação, impacto na arquitetura e estabilidade. Isso permite que a equipe saiba o que precisa ser feito primeiro, quais pontos são mais críticos e quais podem ser deixados para versões futuras.

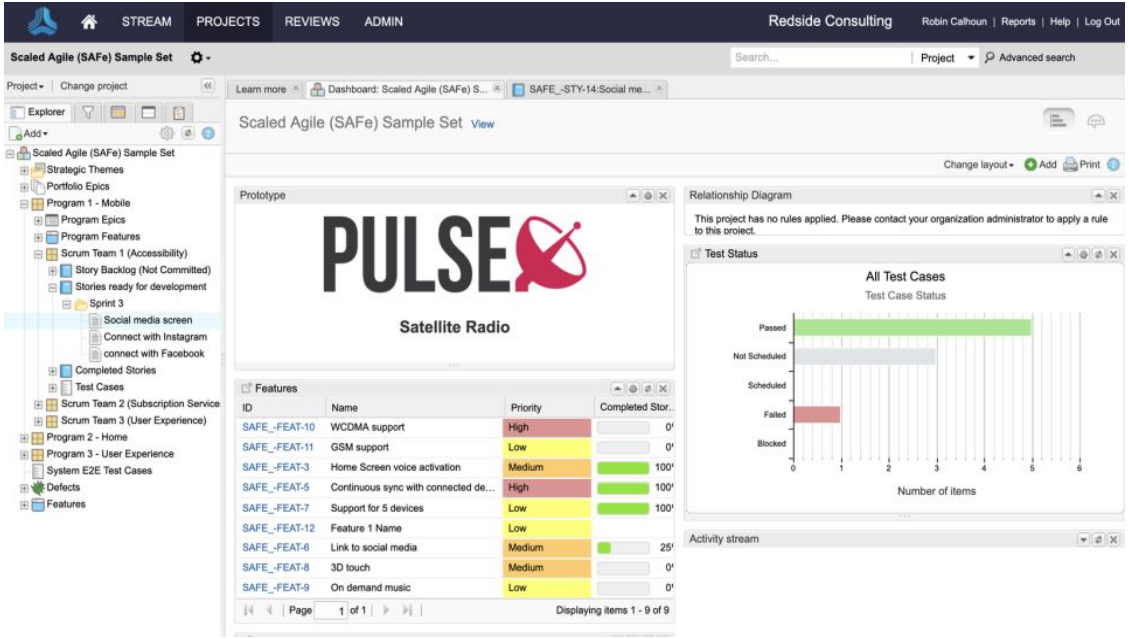
Mas não basta apenas levantar e organizar os requisitos: é preciso **gerenciá-los ao longo de todo o ciclo de vida do software**. O gerenciamento de requisitos garante que eles sejam documentados, acompanhados e adaptados às mudanças que naturalmente surgem durante o projeto. Afinal, raramente um software é construído exatamente como foi planejado no início.

O gerenciamento envolve também a chamada rastreabilidade, que permite acompanhar a evolução de um requisito desde sua origem até a entrega final. Assim, é possível verificar se ele foi realmente **implementado**, se está consistente com os demais e quais impactos podem ocorrer em caso de mudanças.

Durante o **ciclo de vida**, o gerenciamento passa por várias etapas: investigação, quando os requisitos são levantados com usuários, negócio e equipe técnica; análise de viabilidade, que avalia custos e benefícios; projeto, que compara as soluções propostas com os requisitos; construção e testes, que validam se o software atende ao que foi definido; e entrega, que recolhe feedbacks para futuras versões.

As mudanças de requisitos são praticamente **inevitáveis**. Podem ocorrer devido a alterações no negócio, em leis, em tecnologias ou mesmo por falhas no levantamento inicial. Por isso, o gerenciamento de mudanças deve ser sistemático: registrar solicitações, analisar impactos, decidir pela implementação ou não, e atualizar a documentação.

Outro ponto importante é o uso de ferramentas de apoio, que ajudam a organizar, rastrear e versionar requisitos. Existem **softwares específicos**, como o IBM Rational DOORS ou o Jama, mas até planilhas podem ser usadas em projetos menores. O fundamental é manter um controle claro para evitar que informações se percam.



No fim, o grande objetivo da engenharia e do gerenciamento de requisitos é garantir que o sistema **atenda às necessidades** reais dos usuários e do negócio. Isso evita falhas como construir funcionalidades desnecessárias, deixar de lado pontos críticos ou estourar prazos e orçamentos. Com uma boa prática de requisitos, a chance de sucesso do projeto aumenta consideravelmente.

Por exemplo, imagine que uma escola queira um sistema para registrar notas dos alunos. Se o analista apenas ouvir o diretor, talvez o sistema atenda bem à gestão, mas não aos professores. Se só ouvir os professores, pode ignorar relatórios exigidos pela secretaria de educação. Ao aplicar entrevistas com os dois, usar questionários para os alunos e observar como hoje as notas são registradas, o analista terá uma visão completa. Depois, no gerenciamento, será possível priorizar as funções mais urgentes (como o lançamento de notas) e deixar melhorias adicionais (como gráficos de desempenho) para versões futuras.

ATENÇÃO!

A “Situação” ou **status de um requisito** representa em que estágio ele se encontra dentro do ciclo de desenvolvimento de software. No início, um requisito pode estar apenas identificado, ou seja, foi levantado durante a elicitação, mas ainda não foi analisado em detalhes. Depois, ele pode ser priorizado, quando a equipe decide sua importância em relação aos demais. Em seguida, pode ser alocado, ou seja, destinado a uma entrega específica (como uma sprint ou versão). Conforme o desenvolvimento avança, o requisito passa a ser implementado (codificado no sistema), depois testado (verificado para garantir que atende ao esperado), e, se aprovado, é homologado, significando que os stakeholders validaram seu funcionamento. Por fim, o requisito é considerado entregue, quando está disponível na versão final do software. Esse acompanhamento por status é essencial para dar transparência ao andamento do projeto, facilitar o gerenciamento e permitir que todos saibam quais funcionalidades já estão prontas, em progresso ou ainda pendentes.

ENTREVISTA…

“Uma das técnicas mais utilizadas na elicitação de requisitos é a **entrevista**. Ela consiste em um diálogo estruturado entre o analista de sistemas e os stakeholders (usuários, gestores ou clientes), no qual o objetivo é coletar informações sobre as necessidades, expectativas e restrições relacionadas ao sistema que será desenvolvido. Esse método é valioso porque permite aprofundar o entendimento sobre os processos e identificar tanto requisitos explícitos quanto implícitos, além de captar sinais não verbais que revelam prioridades ou preocupações. Por exemplo, em uma empresa de logística que deseja modernizar seu sistema de rastreamento de entregas, o analista pode entrevistar motoristas, atendentes e gerentes. Durante as entrevistas, os motoristas podem destacar a necessidade de um aplicativo simples e acessível durante o trajeto, enquanto os gerentes podem ressaltar a importância de relatórios detalhados sobre atrasos. Dessa forma, a entrevista ajuda a reunir diferentes perspectivas e a compor um conjunto de requisitos mais fiel à realidade da organização…”

OUTRAS TÉCNICAS:

1. Observação - Nessa técnica, o analista acompanha de perto a execução das tarefas dos usuários, observando como o trabalho é realmente feito, sem depender apenas da descrição dos stakeholders. Isso ajuda a identificar requisitos que muitas vezes os próprios usuários não percebem ou esquecem de mencionar.

Exemplo: em um hospital, o analista observa como os enfermeiros registram informações dos pacientes em planilhas impressas. Ele percebe que, além dos dados clínicos, os enfermeiros fazem anotações manuais sobre atrasos na entrega de medicamentos — algo que não havia sido mencionado. Essa observação pode gerar um requisito de “registro digital de ocorrências de medicamentos” no novo sistema.

2. Questionário - É uma técnica eficaz quando há muitos usuários ou quando eles estão distribuídos geograficamente. Permite coletar informações de forma padronizada, mas não oferece a mesma profundidade das entrevistas.

Exemplo: uma rede de escolas deseja implantar um sistema online de notas e presença. Como os professores estão espalhados por diferentes cidades, o analista envia um questionário com perguntas objetivas, como “Quais funcionalidades você considera indispensáveis em um sistema de notas?” ou “Você prefere acessar via computador, celular ou ambos?”. Isso ajuda a identificar necessidades comuns em larga escala.

SUCESSO!

FONTES:
DAVIS, A. M. Software Requirements: objects, functions and states. Englewood Cliffs, New Jersey: Prentice Hall. 1993.
KOTONYA, G.; SOMMERVILLE, I. Requirements Engineering:processes and techniques (worldwide series in computer science). Wiley, 1998.
PFLEEGER, Shari Lawrence. Engenharia de software: teoria e prática. 2. ed. São Paulo: Pearson Education do Brasil, 2007.
PRESSMAN, Roger S. PENTEADO, Rosângela Delloso (Trad.). Engenharia de software. 6. ed. Rio de Janeiro: McGrawHill, 2006.
PRESSMAN, Roger S. Engenharia de software: Uma abordagem Profissional. 7. ed. São Paulo: McGraw-Hill, 2011.
SOMMERVILLE, Ian. Engenharia de software. 9. ed. São Paulo: Pearson Addison Wesley, 2011.
UNIT, USP, Unifacs, Wikipedia, Google Imagens…

manoelmacedo@yahoo.com